

Operating System with Aetherix

A from-scratch guide to x86 operating system and UEFI application development, built around the Aetherix toolkit.



Divyanshu Sinha

License: GNU AGPL-3.0-or-later



Operating System with Aetherix

Author: Divyanshu Sinha

Subject: Aetherix — a cross-platform, from-scratch toolkit for x86 bootloader, kernel, and UEFI application development.

Distributed under the terms of the GNU Affero General Public License, version 3, or (at your option) any later version (AGPL-3.0-or-later). See the final chapter of this book, and the full license text published by the Free Software Foundation, for complete terms.

All source code excerpts in this book are drawn from the Aetherix project and are provided for explanatory purposes under the same license as the project itself.





Table of Contents

1. Preface
2. Introduction to Operating System Development
3. x86 Memory: Slots, Addresses, and Segments
4. The Machine Bytes Behind an x86 Instruction
5. Processor Modes: 16-bit, 32-bit, and 64-bit
6. Firmware: BIOS and UEFI at a Glance
7. How a Bootloader Talks to Hardware
8. Hardware Abstraction Layers (HALs)
9. The Stack and Registers
10. How Aetherix Works Behind the Scenes
11. From Aetherix Code to Machine Code: A Worked Example
12. Kernel Entry and the Boot Sequence
13. AVFS — The Aetherix Volume File System
14. What Is UEFI, and How Does It Work?
15. UEFI in Aetherix: A Separate Subsystem
16. What Makes Aetherix Unique
17. Getting Started: Your First OS with Aetherix
18. Using HALs and Protocols
19. Extending the Ecosystem: Custom HALs and Protocols
20. The Aetherix Program Class: A Practical Reference
21. Writing Code with Program: Worked Examples
22. Contributing to Aetherix
23. License: GNU Affero General Public License v3.0 or later
24. Closing Notes



1. Preface

This book exists for one reason: to explain, from the ground up, how a modern x86 computer actually boots and runs code — and to do it through the lens of a real, working piece of software: Aetherix, a cross-platform toolkit that lets you write bootloaders, kernels, and UEFI applications entirely from Python, with a small C core doing the actual machine-code encoding.

Most operating system tutorials ask you to learn x86 assembly, a linker, a build system, and a handful of arcane tools before you can see a single pixel change on screen. Aetherix takes a different approach: you write Python, and Aetherix's own encoder — not nasm, not gcc, not ld — turns it directly into the bytes the CPU executes. Every layer of that pipeline is something you can read, and this book walks through all of it.

This is not a marketing document. Where Aetherix has real limits — and it does — this book says so plainly. A hobby OS toolkit that pretends to be a finished operating system helps no one. What follows is an honest account of what x86 booting actually involves, and how Aetherix implements the parts of it that are genuinely tractable for one person or a small team to build, test, and understand.

Whether you are here to learn how computers boot, or to actually build something with Aetherix, the chapters are written so you can read start to finish or jump straight to the part you need.

2. Introduction to Operating System Development

An operating system is, at its core, the first program that gets to decide what the computer does with itself. Before your OS runs, there is no concept of files, processes, or even a screen you can print text to in the way a high-level language assumes. There is only a CPU that has just been reset, a fixed entry point defined by the platform (BIOS or UEFI), and raw memory.

Writing an operating system, or even a minimal bootloader, means working at a level where nothing is provided for you. You do not get a `print()` function; you get a video memory address you can write bytes into, or a firmware call you can make while such calls are still available. You do not get `malloc()`; you get a flat address space and whatever memory map the firmware handed you. You do not get threads, or even interrupts, until you build the infrastructure for them yourself.

Why this is hard in practice

The difficulty is rarely conceptual. Setting a video mode, reading a disk sector, or printing a string are not intellectually deep operations. The difficulty is tooling: getting a 512-byte boot sector to assemble to exactly the right bytes, getting a linker script to place your kernel at the address you told the bootloader to jump to, and getting a build pipeline that does not silently produce a broken image. Most people give up during this stage, before they have written a single line of code that does anything interesting.

Aetherix's core design decision is to remove that stage entirely. There is no linker, no separate assembler invocation, and no manual address bookkeeping for the common cases. You describe what you want in Python; Aetherix computes the addresses, resolves the jumps, and assembles the bytes.

Note. Everything in this book was verified against real tools while Aetherix was being built: `objdump` disassembly, CPU emulation (Unicorn Engine) that actually executes the generated bytes, and, for the UEFI chapters, real unmodified OVMF firmware booting the produced disk image in real QEMU. Where a claim in this book could not be verified this way, it is marked as such.

3. x86 Memory: Slots, Addresses, and Segments

Before any instruction executes, the CPU needs an address space to execute it in. On x86, what an address actually means has changed three times across the architecture's history, and an OS developer has to understand all three, because a modern PC still walks through the oldest one on every single boot.

Real-mode addressing: segment:offset

When an x86 CPU is reset, it starts in 16-bit real mode, the same addressing model as the original 8086 from 1978. Memory is addressed with a segment:offset pair. The physical address is computed as:

$$\text{physical_address} = (\text{segment} * 16) + \text{offset}$$

This is why the very first instruction a boot sector executes typically zeroes out the segment registers: BIOS hands off control at physical address 0x7C00 with the segment registers in an unspecified state, and code that assumes DS=0 will read the wrong bytes if that assumption is wrong. A boot sector's first job is almost always to nail down its own addressing environment before doing anything else.

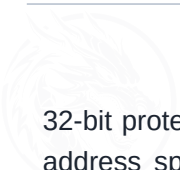
This scheme allows addressing up to about 1 MiB of memory (20-bit physical addresses), which is why the first megabyte of RAM on every PC-compatible machine is still called "conventional memory", decades after it stopped being a practical limit for anything else.

Why an offset like 0x7E00 matters

Aetherix loads a kernel to physical address 0x7E00 by default — immediately after the 512-byte boot sector that starts at 0x7C00. That address is not arbitrary: 512 bytes after 0x7C00 is exactly 0x7E00, and it happens to be paragraph-aligned (divisible by 16), which simplifies later address arithmetic considerably.

A real bug caught during Aetherix's development illustrates why this matters. Real mode's segment:offset addressing has a 16-bit offset. A disk read that tried to load an entire large kernel in a single BIOS call, all at one fixed segment, would need an offset past 0xFFFF once the kernel exceeded roughly 65 sectors (about 33 KB) — something the 16-bit offset field cannot represent. On real hardware this either silently wraps within the same 64 KB segment (corrupting data) or is rejected outright by the BIOS. The fix was to advance the destination by segment rather than by offset between chunks of the read, since a segment can represent a much larger address range than a 16-bit offset alone.

Protected mode and long mode: flat addressing



32-bit protected mode and 64-bit long mode both support a flat memory model: a single, large linear address space with no segment multiplication required. Aetherix kernels build a Global Descriptor Table (GDT) with a flat code and data segment (base 0, limit covering the full addressable range) specifically so that, once the CPU enters protected mode, a memory access like writing to address 0xB8000 (the VGA text buffer) just works, without any segment arithmetic at all.

This is why the transition from real mode to protected mode is really a transition in how the CPU interprets every subsequent memory reference: the exact same 32-bit value in a register means something different in physical memory before and after the GDT is loaded and the mode-switch bit is set.

Kernel-level "variables" without a heap

A kernel with no memory allocator still needs somewhere to keep state that changes at runtime — a text cursor position, a Shift-key flag, a disk buffer. Aetherix's approach, consistent across its BIOS-side drivers, is to reserve fixed physical addresses for these values and treat them exactly like static global variables in a hosted language. The terminal driver's cursor position and column counter, for example, live at fixed addresses just past where a small kernel would plausibly reach, documented directly in the driver's source so a growing kernel knows when it needs to move them.

4. The Machine Bytes Behind an x86 Instruction

Every line of assembly you have ever seen — `mov eax, 4`, or `call [ebx+8]` — is a human-readable label for a specific sequence of bytes. Understanding that encoding is what makes it possible to build an assembler at all, since an assembler is, in the end, a program that maps mnemonics and operands to exactly those bytes.

The general shape of an instruction

A general x86 instruction is built from, in order: optional prefixes, an opcode (one, two, or three bytes), an optional ModR/M byte, an optional SIB byte, an optional displacement, and an optional immediate. Not every instruction uses every field — a bare `ret` is one byte (`0xC3`); a 64-bit move of an absolute address into a register can be ten bytes.

ModR/M: register or memory, decided by three fields

The ModR/M byte is where most of an instruction's addressing flexibility lives. Its eight bits split into three fields: `mod` (2 bits), `reg` (3 bits), and `rm` (3 bits). The `reg` field usually names a register operand (or, for some opcodes, extends the opcode itself); the `rm` field, combined with `mod`, decides whether the other operand is a register or a memory location, and if it is memory, what addressing mode is used.

```
ModR/M byte layout:
bit:  7 6 5 4 3 2 1 0
field: [mod][ reg ][ rm ]

mod = 11 -> rm is a register, direct
mod = 00/01/10 -> rm is a memory operand (no disp / disp8 / disp32)
```

Two specific `rm/mod` combinations carry historical special cases that any encoder has to know about, because they do not mean what the general rule would suggest:

- `rm = 101` (the register numbered 5, i.e. `RBP` or `R13`) with `mod = 00` does not mean "[`RBP`]" with no displacement" — it means "no base register, `disp32` only", a holdover from the 8086's addressing modes.
- `rm = 100` (the register numbered 4, i.e. `RSP` or `R12`) always means "a SIB byte follows", regardless of `mod`. This is not optional: encoding [`RSP`] or [`R12`] as a base register without a SIB byte produces a different, wrong instruction.

Note. This second case caused a genuine bug during Aetherix's UEFI development: a generic addressing helper correctly handled the RBP/R13 case but did not emit a SIB byte for RSP/R12, so using R12 to hold a located protocol's interface pointer — an entirely natural choice — silently corrupted the instruction stream. It was caught because a test that used R12 crashed instead of quietly producing wrong output, and the fix (always emit the SIB byte when the base register's low three bits are 100) is now applied uniformly.

SIB: scale, index, base

When a SIB byte is present, it encodes a more general addressing form: $\text{base} + (\text{index} * \text{scale})$. Its fields are scale (2 bits, giving a multiplier of 1, 2, 4, or 8), index (3 bits, the register to scale), and base (3 bits, the register to add unscaled). Aetherix's own encoder does not yet use scaled-index addressing — its addressing helpers stick to $\text{base} + \text{displacement}$ — but it does emit the "no index" SIB encoding (index field = 100) whenever RSP or R12 is used as a plain base register, exactly because of the special case above.

REX: the prefix that makes 64-bit mode possible

Long mode introduced eight more general-purpose registers (R8–R15) and the ability to use a full 64-bit operand size, and it did this without changing any existing opcode. Instead, a new one-byte prefix, REX, was added. Its four significant bits are:

```

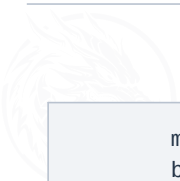
REX byte: 0100 W R X B
W = 1 selects a 64-bit operand size instead of the instruction's default
R = extends the ModR/M 'reg' field (adds a 4th, high bit)
X = extends the SIB 'index' field
B = extends the ModR/M 'rm' field or an opcode-embedded register

```

Concretely, this is why `mov r8, rax` needs a REX prefix with only the B bit set (0x41): the base opcode for "move register to register/memory" is unchanged from 32-bit mode, but the destination register number (8) needs one more bit than the three the ModR/M `rm` field provides, and REX.B supplies it.

Worked example: three ways to move a value

To make the encoding concrete, here are three related instructions and their exact byte encodings, as Aetherix's own encoder produces them and as `objdump` independently confirms them:



```
mov eax, 0x1234 (32-bit, zero-extends into RAX in 64-bit code)
bytes: B8 34 12 00 00
B8+reg selects EAX; the next 4 bytes are the little-endian immediate

mov rax, 0x1234567890 (64-bit, full 8-byte immediate)
bytes: 48 B8 90 78 56 34 12 00 00 00
48 is REX.W; B8+reg is the same opcode, now widened by REX.W;
the immediate is a full 8 bytes

mov rax, QWORD PTR [r9+0x20]
bytes: 49 8B 41 20
49 is REX.W + REX.B (R9 needs the extra bit as a base register);
8B is 'load into register'; 41 is ModR/M (mod=01, reg=000, rm=001);
20 is the 1-byte displacement
```

Every one of these encodings is generated, byte for byte, by Aetherix's native encoder (native/encoder.c) rather than by an external assembler, and every one was confirmed correct by disassembling the actual output with objdump during development.



5. Processor Modes: 16-bit, 32-bit, and 64-bit

An x86-64 CPU can operate in three fundamentally different instruction-decoding modes, and a from-scratch OS toolkit has to generate genuinely different byte sequences for each one, because the same bytes can mean different instructions depending on which mode is active.

Real mode (16-bit)

This is the mode every x86 PC boots into, unconditionally, for backward compatibility with software written for the original 8086. Registers default to 16-bit width, addressing is segment:offset, and there is no memory protection whatsoever — any code can read or write any address, including the interrupt vector table at address 0. BIOS services (disk reads, video output, keyboard polling) are only available here, accessed via the INT instruction, which is why every BIOS-based bootloader's first job is done entirely in this mode.

Protected mode (32-bit)

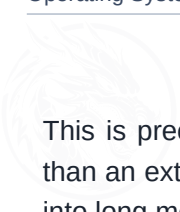
Entering protected mode requires three things: a Global Descriptor Table describing the segments available, the LGDT instruction to load it, and setting bit 0 (the Protection Enable bit) of control register CR0. Immediately after, a far jump is required to actually reload the code segment register with a protected-mode selector — simply setting the CR0 bit does not retroactively change how the currently executing code segment is interpreted.

```
; The core real-mode-to-protected-mode transition, as Aetherix generates it:
cli ; disable interrupts -- nothing is set up to handle them yet
lgdt [gdt_address] ; load the Global Descriptor Table register
mov eax, cr0
or eax, 1 ; set PE (Protection Enable)
mov cr0, eax
jmp 0x08:protected_entry ; far jump: reloads CS with a flat 32-bit code selector
```

With a flat GDT (base 0, 4 GiB limit, for both code and data segments), every subsequent memory access behaves as if segmentation did not exist: the value in a 32-bit register is the physical address, full stop.

Long mode (64-bit)

Long mode is reached from protected mode, and additionally requires paging to be enabled — unlike protected mode, long mode has no concept of running without page tables. Aetherix's BIOS-side kernel path does not enter long mode; its 32-bit kernels run their entire lifetime in protected mode. UEFI applications, in contrast, are handed control by firmware already in long mode, with paging already configured — the firmware has done that setup before your code ever runs.



This is precisely why Aetherix's UEFI support is a separate encoder path (Program(bits=64)) rather than an extension of the BIOS path: there is no 16-to-64 transition to generate, because the transition into long mode already happened before a UEFI application's entry point is called.

One encoder, three modes, and why that matters

Aetherix represents this with a single Program class parameterized by a bits argument (16, 32, or 64). This is not a cosmetic choice: several opcodes are byte-identical across modes (a near jmp with a 32-bit relative offset, for instance, is encoded the same way in 32-bit and 64-bit code), while others critically are not. A 16-bit mov reg, imm16 and a bare mov reg, imm32 in 32-bit code share the same base opcode (0xB8 + register number) but differ in how many immediate bytes follow; using the wrong width's encoder function inside the wrong mode silently desynchronizes the rest of the instruction stream. Aetherix's own development caught exactly this mistake once: a helper that set a 16-bit port number worked fine inside 16-bit real-mode code but corrupted subsequent instructions when reused, unmodified, inside 32-bit protected-mode code, because the same opcode defaults to a 32-bit immediate there.

6. Firmware: BIOS and UEFI at a Glance

"Firmware" is the software permanently built into the motherboard that runs before any operating system, and its job is to initialize hardware to a known state and then hand control to whatever is on the boot device. Two competing standards exist for this hand-off on PC-compatible hardware, and Aetherix supports both, as two entirely separate pipelines.

BIOS: the legacy path

The Basic Input/Output System hands off control by loading the first 512 bytes of the boot device to physical address 0x7C00 and jumping to it, provided those 512 bytes end with the signature bytes 0x55 0xAA. Everything about disks, video, and keyboards is accessed through software interrupts (the INT instruction) while still in 16-bit real mode; there is no other API. This is simple to reason about and easy to build tooling for, which is exactly why Aetherix's BootSector/Kernel path targets it directly.

UEFI: the modern path

The Unified Extensible Firmware Interface takes a completely different approach: instead of loading a raw 512-byte sector, it loads and executes a properly formatted PE32+ executable — the same file format Windows uses for 64-bit programs, just with a different subsystem field — directly in 64-bit long mode, with paging already enabled. It finds this file by reading a GPT partition table, locating a partition of a specific type (the EFI System Partition), mounting it as a FAT filesystem, and looking for a file at a fixed path: \EFI\BOOT\BOOTX64.EFI on removable media with no other boot entry registered.

Instead of BIOS interrupts, UEFI exposes services as function pointers reachable through a structure called the `EFI_SYSTEM_TABLE`, using the Microsoft x64 calling convention regardless of the host operating system. Text output, for example, goes through `SystemTable->ConOut->OutputString(This, String)` — a genuine function call through a pointer, not a software interrupt.

Why Aetherix treats these as two unrelated pipelines

There is essentially no overlap between what a BIOS bootloader and a UEFI application need to do. One deals in 16-bit real mode, MBR sectors, and interrupts; the other deals in 64-bit long mode, PE executables, GPT partitions, and protocol function pointers. Aetherix reflects this honestly in its code layout: `aetherix.context/boot/kernel/drivers` form the BIOS pipeline, and `aetherix.uefi` is an entirely separate subsystem, built from scratch, sharing nothing but the underlying philosophy of "emit real bytes yourself, verify them independently".

7. How a Bootloader Talks to Hardware

A bootloader's entire purpose is to get a kernel into memory and transfer control to it, and doing that means talking directly to hardware — through BIOS services while they are still available, and through raw I/O ports once they are not.

BIOS interrupts as a hardware protocol

Calling a BIOS service means setting specific registers to specific values and issuing an INT instruction with a specific interrupt number; the combination of interrupt number and an initial value (often in the AH register) selects the operation. This is, in effect, a very old but very stable calling convention, and an OS toolkit has to encode it exactly.

Reading disk sectors is the clearest example. The modern, geometry-independent way is the INT 13h "extended read" function (AH=0x42), which takes a pointer to a 16-byte Disk Address Packet describing how many sectors to read, where to put them in memory, and which logical block address to start from:

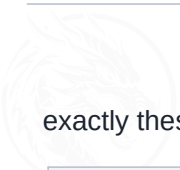
```
Disk Address Packet (16 bytes):  
offset 0 size of packet (0x10)  
offset 1 reserved (0)  
offset 2 sector count (word)  
offset 4 destination offset (word)  
offset 6 destination segment (word)  
offset 8 starting LBA, low 32 bits  
offset 12 starting LBA, high 32 bits
```

Aetherix's bootloader builds this packet, points SI at it, sets AH=0x42, and issues INT 0x13 — and, critically, does this in chunks no larger than 64 sectors. A single oversized read would need a destination offset past 0xFFFF, which a 16-bit register cannot represent; Aetherix instead advances the destination by segment between chunks, since a segment value has much more range than an offset alone.

I/O ports: hardware without BIOS

Not every device is reachable through a BIOS call, and once a kernel is running in protected mode, BIOS calls are not available at all (they assume 16-bit real mode). Direct hardware access on x86 happens through I/O ports, read and written with the IN and OUT instructions, addressed by a 16-bit port number.

The PS/2 keyboard controller is a good example of a protocol built entirely out of two ports: writing to port 0x64 sends a command to the controller itself, reading from port 0x64 returns a status byte (bit 0 indicates whether a scancode is waiting), and port 0x60 is where the actual scancode is read from or a command byte written to. A keyboard driver is, underneath, a small state machine built on top of



exactly these two ports:

```
; Busy-poll for one keyboard scancode
poll_loop:
in al, 0x64 ; read PS/2 controller status
test al, 0x01 ; bit 0: output buffer full?
jz poll_loop ; not yet -- keep waiting
in al, 0x60 ; read the scancode itself
```

A second example is the classic PC speaker: setting the PIT (Programmable Interval Timer) to a given frequency and toggling two bits of port 0x61 makes an audible tone, entirely through port I/O, with no BIOS involvement and no dedicated audio hardware required.

Hardware reset without an operating system

Even something as blunt as rebooting the machine is a hardware protocol: writing the value 0xFE to the keyboard controller's command port (0x64) pulses the CPU's reset line, a technique that predates any concept of an OS-level reboot API and still works identically on real hardware and every mainstream emulator today.

8. Hardware Abstraction Layers (HALs)

A Hardware Abstraction Layer exists to answer one question cleanly: what hardware capabilities does this kernel actually have available, and where is the code that implements each one? Without a HAL, a kernel's driver code and its higher-level logic tend to become inseparable, and it becomes hard to tell, just by reading a kernel's source, what it can and cannot do.

Aetherix's HAL as a registry, not a promise

Aetherix's HAL (`aetherix.drivers.hal.HAL`) is a small registry mapping a name (such as "vga" or "keyboard") to a driver module. What makes it distinctive is that it is explicit about what is real: calling `HAL().implemented()` lists drivers that are genuinely working code, and `HAL().scaffolded()` lists named extension points for hardware that is not implemented yet — a modern GPU framebuffer, USB, networking, and so on. Accessing a scaffolded driver raises a clear `NotImplementedError` explaining exactly what it would need, rather than silently doing nothing or pretending to work.

```
from aetherix import HAL
hal = HAL()
print(hal.implemented())
# ['vga', 'keyboard', 'speaker', 'terminal', 'graphics', 'power']
print(hal.scaffolded())
# ['gpu', 'usb', 'ethernet', 'printer', 'fan', 'battery', 'camera']
```

What is genuinely implemented, and why those specific things

Every driver Aetherix ships targets hardware that is reachable by writing directly to fixed memory addresses or I/O ports, without needing bus enumeration, a protocol stack, or another driver underneath it first:

- vga — the VGA text-mode framebuffer at physical address 0xB8000 (an 80x25 grid of character+attribute byte pairs), and VGA Mode 13h, a 320x200, 256-color linear framebuffer at 0xA0000, reachable with a single BIOS mode-set call.
- keyboard — PS/2 controller polling, covering a full Scancode Set 1 table (letters, digits, punctuation, Tab) with Shift-state tracking for uppercase and shifted symbols.
- speaker — the PC speaker, driven through the Programmable Interval Timer and port 0x61.
- terminal — a runtime VGA text cursor built on top of vga, since vga's own primitives only place compile-time-known text at compile-time-known positions.
- graphics — palette upload and image blitting for VGA Mode 13h, including a build-time image preparation module for ordinary PNG/JPEG files.

- power — a soft restart (jump back to the kernel's own entry point), a hard reboot (the keyboard-controller reset pulse described in the previous chapter), and an honestly-scoped shutdown that only works on QEMU/Bochs (real ACPI shutdown needs parsing ACPI tables, which is not implemented).

Extending the HAL yourself

Because the HAL is a plain registry, adding a driver does not require touching Aetherix's own source at all. A driver module is just a set of functions that take a Program and append instructions to it, following the same shape as the built-in drivers:

```
from aetherix.drivers.hal import HAL
import my_serial_driver # your own module, shaped like vga.py/keyboard.py

hal = HAL()
hal.register("serial", my_serial_driver)

from aetherix import Project
proj = Project("MyOS", hal=hal)

@proj.kernel_entry
def main(prog, drivers):
    drivers.serial.write_byte(prog, ord('X'))
```

This is deliberate: the HAL's job is to make what exists legible, not to gatekeep what is possible. CONTRIBUTING.md documents a concrete, ordered list of realistic next drivers — a serial port, a parallel-port printer, PCI enumeration — specifically so a new contributor has a tractable starting point rather than facing the whole scaffolded list at once.

9. The Stack and Registers

Registers and the stack are the only storage a CPU has before an operating system sets anything else up, and understanding exactly what a kernel can rely on, and when, is a recurring theme in low-level development.

General-purpose registers across modes

The same physical registers are addressed differently depending on operand size: AX is the low 16 bits of EAX, which is the low 32 bits of RAX, and so on for the other seven original registers. Long mode adds eight entirely new registers, R8 through R15, which did not exist before 64-bit mode at all. Aetherix's register constants (`aetherix.regs`) reuse the same numbering (0 through 7) across its 16-bit, 32-bit, and 64-bit encoders precisely because the underlying register is the same physical storage; only the encoder's bits setting changes which width is active.

The stack: push, pop, and calling convention

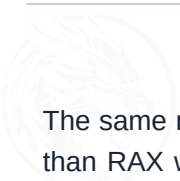
The stack grows downward in x86 — pushing a value decreases the stack pointer (SP/ESP/RSP) before writing to it, and popping increases it after reading. A `CALL` instruction implicitly pushes a return address; `RET` implicitly pops it back into the instruction pointer. This single mechanism is what makes function calls possible at all below the level of any language runtime.

What a function is allowed to assume about the stack, and about which registers survive a call, is entirely a matter of convention — the hardware enforces none of it. Two conventions matter throughout this book:

- **Stack alignment:** the Microsoft x64 calling convention that UEFI uses (regardless of host OS) requires the stack pointer to be 16-byte aligned immediately before a `CALL` instruction. A UEFI application's entry point is itself called this way, so its stack pointer is 16-aligned minus 8 (the just-pushed return address) at the very first instruction — one push, and it is realigned to 16.
- **Shadow space:** the same convention requires the caller to reserve 32 bytes of stack space before every call, whether or not the callee actually uses it. Aetherix's UEFI protocol-call helper does exactly this — `sub rsp, 0x20` before every call, `add rsp, 0x20` after — automatically, so user code never has to remember it.

Callee-saved registers, and a subtle bug they explain

Some registers are conventionally preserved across a function call (callee-saved) and some are not (caller-saved, or "volatile"). Under the Microsoft x64 convention, RBX, RBP, RDI, RSI, and R12–R15 are callee-saved; RAX, RCX, RDX, R8, R9, R10, and R11 are not. This is why Aetherix's `UefiApp` saves the `EFI_SYSTEM_TABLE` pointer handed to it at entry into RBX rather than leaving it in RDX: RDX is volatile, so the very first protocol call the application makes would be free to overwrite it.



The same reasoning is why Aetherix's console helpers hold the ConOut protocol pointer in RSI rather than RAX while making a call — RAX is used internally to hold the function pointer being called, so using it to also hold a value that needs to survive the call is a genuine conflict, not a style preference.

10. How Aetherix Works Behind the Scenes

This chapter is about the machinery: what actually happens between writing `prog.mov16(regs.AX, 0x07C0)` in Python and a file on disk containing the exact bytes `B8 C0 07`.

The native encoder: one C function per instruction shape

At the very bottom of Aetherix sits `native/encoder.c`, a small C file with no dependency on any assembler, linker, or external toolchain. Each function in it encodes exactly one instruction shape — for example:

```
AETH_API void aeth_mov_r16_imm16(enc_buf_t *b, uint8_t reg, uint16_t imm) {  
    push8(b, 0xB8 + (reg & 7));  
    push16(b, imm);  
}
```

This is compiled once, at install time (or on first import, for a development checkout), into a small shared library. Python talks to it through `ctypes` — a standard-library bridge that lets Python call a C function directly, with no additional runtime or build step.

`asm.py`: the assembler layer

A single native function call only produces one instruction's bytes; it has no concept of labels, jumps, or where in a larger program it sits. That is the job of `aetherix.asm.Program`, which records a sequence of instruction "nodes" (a fixed byte size and a function that emits those bytes) rather than emitting bytes immediately.

This two-pass design is what allows a program to jump both forward and backward to labels defined anywhere else in the same code, including labels defined later in the source than the jump that targets them:

```
def assemble(self) -> bytes:
    # Pass 1: every node's size is already known, so offsets can be
    # computed by simple arithmetic -- no native calls needed yet.
    labels = {}
    offset = 0
    for node in self._nodes:
        if node.label is not None:
            labels[node.label] = offset
            offset += node.size

    # Pass 2: labels are all known now, so every jump/call/lea can
    # compute its real relative offset and emit for real.
    buf = NativeBuffer()
    offset = 0
    for node in self._nodes:
        if node.label is None:
            node.emit(buf, offset, labels)
            offset += node.size
    return buf.to_bytes()
```

This only works because every instruction's size is independent of the values it will eventually hold — a near jump is always 5 bytes in 32-bit code whether its target is 3 bytes away or 3000. That invariant is enforced by convention across every native encoder function, and is documented as a hard rule for anyone adding a new one.

Position independence, without a linker

Aetherix's UEFI code generation goes one step further: it never emits an absolute address pointing at its own code or data. Every reference to embedded data uses RIP-relative addressing (`lea reg, [rip + offset]`), and every jump or call to a label in the same program uses a relative offset. Because of this, the resulting machine code is correct no matter where in memory it is actually loaded — which is precisely why Aetherix's PE32+ writer can omit the `.reloc` section that a conventionally linked executable would need. There is nothing to relocate.

11. From Aetherix Code to Machine Code: A Worked Example

To make the whole pipeline concrete, this chapter follows one small piece of code from Python source to verified machine bytes.

The source

```
from aetherix.asm import Program
from aetherix import regs

p = Program(bits=16)
p.cli()
p.xor16(regs.AX, regs.AX)
p.mov_sreg(regs.DS, regs.AX)
p.mov16(regs.SP, 0x7C00)
p.sti()
code = p.assemble()
```

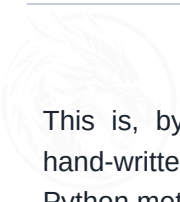
What each call actually does

- `p.cli()` appends a one-byte node: opcode `0xFA`, no operands.
- `p.xor16(AX, AX)` appends a two-byte node: opcode `0x31` followed by a ModR/M byte computed from the two register numbers (both 0 for AX here), giving `0xC0`.
- `p.mov_sreg(DS, AX)` moves the (now zero) value in AX into the DS segment register: opcode `0x8E` plus a ModR/M byte encoding the segment/register pair.
- `p.mov16(SP, 0x7C00)` appends a three-byte node: `0xB8` + register number for SP, followed by the two-byte little-endian immediate `0x7C00`.
- `p.sti()` appends a one-byte node: opcode `0xFB`.

The result, independently verified

Calling `p.assemble()` runs the two-pass process described in the previous chapter and returns the raw bytes: `FA 31 C0 8E D8 BC 00 7C FB`. Disassembling that exact byte sequence with `objdump`, entirely independently of Aetherix, confirms it reads back as:

```
0: fa cli
1: 31 c0 xor ax,ax
3: 8e d8 mov ds,ax
5: bc 00 7c mov sp,0x7c00
8: fb sti
```



This is, byte for byte, the standard segment-initialization prologue that opens essentially every hand-written x86 boot sector ever taught in an OS development course — produced here entirely from Python method calls, with no assembler invoked at any point.

12. Kernel Entry and the Boot Sequence

This chapter traces the full sequence from BIOS handing off control to a kernel's own code running, which is where Aetherix's BootSector and Kernel classes meet.

Stage 1: the boot sector

`BootSector.standard()` builds a complete 512-byte boot sector: segment setup, an optional status message printed through the BIOS teletype call, enabling the A20 address line (needed for correctness once addresses beyond the first megabyte come into play), loading the kernel from disk via the extended-read mechanism described earlier, and finally a far jump into the loaded kernel. The very last two bytes are always 0x55 and 0xAA, without which BIOS will not treat the sector as bootable at all.

Stage 2: the kernel's own trampoline

The kernel image Aetherix produces is not purely 32-bit code — it begins with a short 16-bit trampoline, because the boot sector hands off control while the CPU is still in real mode. This trampoline builds a flat GDT, sets the CR0 protection bit, and far-jumps into the 32-bit body, exactly as described in the chapter on processor modes. Its layout is:

```
[ 16-bit trampoline ][ 6-byte GDTR ][ 24-byte GDT table ][ 32-bit kernel body ]
```

Because the trampoline only ever uses fixed-width instructions, its total size is a compile-time constant, which means the addresses of the GDTR, GDT table, and the 32-bit body can all be computed analytically before a single trampoline byte is emitted — no forward-reference patching is needed even here.

@kernel_entry: where user code begins

The `@proj.kernel_entry` decorator marks the Python function that becomes the kernel's actual logic — the first thing that runs once the 32-bit body starts executing. `Kernel.assemble()` places a label at the very start of that body before calling the decorated function, specifically so that a "restart" primitive (jump back to the kernel's own start) is always available without the user having to manage that label themselves.

```
from aetherix import Project

with Project("MyOS") as os:

    @os.kernel_entry
    def main(prog, drivers):
        drivers.vga.clear(prog)
        drivers.vga.print_string(prog, "Hello, world!")
        prog.hlt()

    os.build("myos.img")
```

Everything after that point — clearing the screen, printing a message, halting — is just further Program instructions, generated by whichever driver functions the kernel calls, exactly as described in the HAL chapter.

The halt loop, and why it matters

`Kernel.assemble()` appends a defensive infinite halt loop after the user's entry function, unless explicitly disabled. Falling off the end of a kernel's code is not a graceful event — the CPU will simply continue executing whatever bytes come next, which is normally uninitialized memory or unrelated data. A halt loop turns that failure mode into a predictable, harmless stop instead.

13. AVFS — The Aetherix Volume File System

A kernel with no filesystem driver still frequently needs to ship data alongside its code — a font, a configuration file, a small asset. AVFS exists to solve exactly that problem, and nothing more.

What AVFS is, precisely

AVFS is a deliberately minimal, read-only, build-time file archive. Files are embedded into a disk image when it is built, using a layout simple enough to parse from 16-bit or 32-bit kernel code without a directory tree, permissions, or free-space management — none of which a build-time-only archive needs.

```
AVFS layout:
offset 0 magic 4 bytes b"AVFS"
offset 4 version 1 byte currently 1
offset 5 file_count 1 byte up to 255 files
offset 6 entries file_count * 40 bytes, each:
name 32 bytes UTF-8, NUL-padded
offset 4 bytes little-endian, from archive start
size 4 bytes little-endian, in bytes
... file data back-to-back, in entry order
```

Building an archive

```
from aetherix import Archive

archive = Archive()
archive.add_file("readme.txt", "Hello from disk!\n")
archive.add_path("assets/logo.bin")

blob = archive.build() # bytes: header + file table + raw file data
```

`Project.add_file()` and `Project.add_path()` do this automatically and append the resulting archive after the kernel's own sectors when the disk image is built, so most users never construct an `Archive` object directly.

What AVFS deliberately does not do

There is currently no kernel-side code in Aetherix that reads an AVFS archive back out at runtime. This is stated plainly rather than glossed over: AVFS today is purely a build-time embedding mechanism, comparable to an `initrd` in spirit, and writing the kernel-side parser that turns it into something a running OS can open files from is exactly the kind of "further OS development" contribution the project's own documentation points newcomers toward. The format's simplicity is deliberate preparation for that: a linear scan of 40-byte entries is well within reach of hand-written 16-bit or 32-bit code, once someone writes it.

14. What Is UEFI, and How Does It Work?

UEFI (Unified Extensible Firmware Interface) is the successor to BIOS on virtually all PC hardware sold since roughly the early 2010s, and it changes almost every assumption a BIOS-era bootloader relies on.

From power-on to your code

When a UEFI machine powers on, firmware initializes hardware, then walks a list of configured boot options (or, for removable media with no configured entry, looks for a fixed default path). For each candidate, it needs to find a GPT-partitioned disk, locate the partition whose type GUID marks it as an EFI System Partition, mount that partition's FAT filesystem, and load a PE32+ executable from it — `\EFI\BOOT\BOOTX64.EFI`, in the removable-media case.

GPT: partitioning without MBR's limits

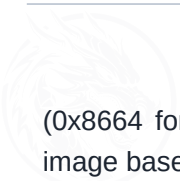
The GUID Partition Table replaces the decades-old MBR partitioning scheme, which could only address a limited number of partitions with 32-bit sector counts. A GPT disk still begins with a "protective MBR" — a single MBR-style partition entry marking the whole disk as an unrecognized type, so that tools which only understand MBR do not mistake the disk for empty and offer to overwrite it. The real partition table follows: a header (with a CRC32 checksum over itself) and an array of partition entries, each also identified by GUID rather than a one-byte type code. The spec requires a complete backup copy of both the header and the entry array at the very end of the disk, and firmware may refuse to boot a disk whose backup does not validate.

FAT32: still relevant, decades later

The EFI System Partition is formatted as FAT (FAT32, for anything but the smallest media), the same filesystem format from MS-DOS, still in active use specifically because it is simple, universally implemented, and requires no licensing negotiation. Whether a given FAT volume is really "FAT32" as opposed to FAT16 is not determined by an explicit flag — it is determined by the number of data clusters the volume's own boot sector parameters imply, following Microsoft's own published algorithm. A FAT32 volume needs at least 65,525 clusters to be unambiguously recognized as such by strict tools; building a small volume with 512-byte clusters (rather than the more common 4 KiB) keeps the on-disk size modest while still comfortably clearing that threshold.

PE32+: the executable format itself

UEFI applications are Portable Executable files — the same container format Windows uses — with the Subsystem field in the optional header set to the value 10 (`EFI_APPLICATION`) rather than a Windows subsystem. A minimal valid file needs a DOS header (mostly vestigial, but its magic bytes and the offset to the real header are both checked), a COFF file header naming the target machine



(0x8664 for x86-64) and section count, a PE32+ optional header carrying the entry point address, image base, and subsystem, and at least one section header describing where the actual code lives in the file.

The calling convention, and why it matters even off Windows

A UEFI application's entry point is called as `UefiMain(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable)`, using the Microsoft x64 calling convention — the first four integer arguments in RCX, RDX, R8, and R9, a caller-reserved 32-byte shadow space before every call, and a 16-byte-aligned stack at the point of any call. This is a UEFI specification requirement, not a Windows-specific detail: it applies identically on a Linux machine running OVMF firmware, because the calling convention is a property of the UEFI environment itself, not of any operating system running under it.

Boot Services versus Runtime Services

Everything a UEFI application typically uses — console I/O, memory allocation, protocol lookup — is a Boot Service, available only until the application (or an OS loader built on top of one) calls `ExitBootServices`. After that call, firmware hands the running code a memory map and gets out of the way almost entirely, leaving only a small set of Runtime Services (mainly for things like the real-time clock and NVRAM variables) still callable. A UEFI application that never calls `ExitBootServices` — the kind Aetherix builds — simply uses Boot Services throughout its short life and returns control to firmware when it finishes, the same way a shell command returns to its shell.

15. UEFI in Aetherix: A Separate Subsystem

Aetherix's `aetherix.uefi` package is not a variant of its BIOS pipeline; it is a complete second implementation, sharing only the underlying philosophy of generating real bytes and verifying them independently.

The 64-bit encoder

A minimal but genuinely correct 64-bit long-mode instruction set: REX-prefixed moves, arithmetic, and comparisons; RIP-relative `lea` for position-independent data references; and indirect calls through a register, which is how every UEFI protocol function pointer gets invoked. It supports the full register range, RAX through R15, which matters in practice: a call like `SetCursorPosition(This, Column, Row)` needs a third argument, which the calling convention places in R8.

UefiApp: the entry point contract

```
from aetherix.uefi.app import UefiApp
from aetherix.uefi import console

app = UefiApp()

@app.entry
def main(prog, sys_table):
    console.clear_screen(prog, sys_table)
    console.println(prog, "Hello from Aetherix UEFI!", sys_table)

efi_path = app.build("BOOTX64.EFI")
```

`UefiApp` generates the prologue this entry point needs (saving the `SystemTable` pointer into `RBX`, a callee-saved register, before any user code runs) and an epilogue that returns `EFI_SUCCESS` once the user's function falls through, wraps the result in a real PE32+ file via `aetherix.uefi.pe`, and needs no external linker to do it.

Console and keyboard, by structure offset

`aetherix.uefi.console` and `aetherix.uefi.keyboard` call directly through the `EFI_SYSTEM_TABLE` and its `ConOut/ConIn` protocol structures, at their spec-defined byte offsets (`ConOut` at offset `0x40`, `OutputString` at offset `0x08` within it, and so on). Keyboard input is a busy-poll of `ConIn->ReadKeyStroke`, matching the same design used on the BIOS side, and for the same reason: a genuinely interrupt-driven wait needs an event/IDT infrastructure this project has not built yet.

The extension mechanism: protocol and boot_services

Console and keyboard access cover exactly two UEFI protocols. Everything else in the UEFI specification — Graphics Output Protocol, Block I/O, Simple File System, or any protocol Aetherix

does not wrap — is reachable through two general-purpose functions: `boot_services.locate_protocol()`, which finds a protocol interface by its GUID, and `protocol.call()`, which invokes an arbitrary method on it by byte offset, correctly handling the Microsoft x64 argument registers either as a COM-like interface call (This passed implicitly) or a plain Boot Services table call (no self-pointer at all).

```
from aetherix.uefi import boot_services, protocol
from aetherix import regs

boot_services.locate_protocol(prog, sys_table,
    "9042A9DE-23DC-4A38-96FB-7ADE080516A", regs.R12)
prog.cmp64(regs.EAX, 0)
prog.jcc(regs.JNZ, "not_found")
protocol.call(prog, regs.R12, 0x30, args=[("imm", 42)])
```

This mirrors the BIOS-side HAL's philosophy — make the escape hatch a first-class, documented mechanism rather than an afterthought — adapted to how UEFI actually structures its capabilities.

The disk image: GPT and FAT32, from nothing

`aetherix.uefi.gpt`, `fat32`, and `diskimage` together build a complete bootable disk image containing exactly one file, `\EFI\BOOT\BOOTX64.EFI`, with a correct protective MBR, correct primary and backup GPT structures (CRC32 checksums included), and a correct FAT32 volume — all written in pure Python, since constructing these formats is data serialization, not machine-code generation.

How all of this was actually verified

Every layer was checked with a tool that owes Aetherix's own logic nothing: `file` identifies the produced `.efi` as a genuine PE32+ EFI application; `objdump` disassembles the machine code independently; a Unicorn Engine (UC_MODE_64) harness simulates just enough of `EFI_SYSTEM_TABLE`, `ConOut`, `ConIn`, and Boot Services to execute the real generated bytes and confirm the exact right protocol calls happen with the exact right arguments; `gdisk` validates the GPT structure directly ("No problems found"); `fsck.vfat` and `mttools` validate the FAT32 filesystem, including extracting the embedded file back out and confirming it is byte-for-byte identical to the original; and, as the final and strongest check, real, unmodified OVMF firmware, in real QEMU, booted the produced disk image and printed exactly the expected text before returning cleanly to firmware.

16. What Makes Aetherix Unique

Plenty of tutorials exist for writing a boot sector by hand in nasm. What distinguishes Aetherix is a specific combination of choices, each defensible on its own, that together produce something genuinely different from "assembly, but with extra steps".

- No external assembler or linker, at any point. The C core in native/encoder.c is the only thing that turns an instruction description into bytes; there is no nasm, gas, or ld invoked anywhere in the pipeline, for either the BIOS or the UEFI path.
- A genuine, from-scratch implementation of both major boot architectures — BIOS and UEFI — rather than one, with the honest acknowledgment that they share almost nothing at the byte level and should not be forced into a single API.
- A Python-first API, so building a bootable image is writing Python functions, not hand-assembling opcodes — while the underlying Program class still gives full, byte-level control to anyone who wants it, with no hidden abstraction in the way.
- Honesty about scope, enforced structurally rather than just claimed in prose: the HAL explicitly separates implemented drivers from named-but-unbuilt extension points, and functions like `power.shutdown()` document exactly which platforms they work on and why, rather than presenting a false sense of portability.
- A verification culture that treats "it disassembles correctly" as necessary but not sufficient. Every subsystem in this book was checked by actually executing the generated code (via CPU emulation) or by validating the produced file format with an independent, real-world tool — and, for UEFI, by booting the result on real, unmodified firmware.
- A track record, visible in this book, of catching its own real bugs during development — a disk-loading overflow, a register-clobber mistake, a missing SIB byte — specifically because that verification discipline was applied consistently rather than assumed to be unnecessary once something "looked right".

None of this makes Aetherix a finished operating system, and it does not claim to be one. What it offers is a considerably lower barrier to the part of OS development that is usually the hardest to get started with — producing a correct, bootable image at all — without hiding what is actually happening at the byte level once you get there.

17. Getting Started: Your First OS with Aetherix

Installing

```
pip install aetherix
# core. Nothing beyond that is needed to build images -- no nasm, no ld.

pip install "aetherix[imaging]" # optional: image support for graphics.py, needs
Pillow
```

A minimal BIOS-based OS

```
from aetherix import Project

with Project("MyOS") as os:

    @os.kernel_entry
    def main(prog, drivers):
        drivers.vga.clear(prog)
        drivers.vga.print_string(prog, "Hello, world!")
        prog.hlt()

    os.build("myos.img")
```

```
qemu-system-i386 -drive file=myos.img,format=raw
```

A minimal UEFI application

```
from aetherix.uefi.app import UefiApp
from aetherix.uefi import console, diskimage

app = UefiApp()

@app.entry
def main(prog, sys_table):
    console.clear_screen(prog, sys_table)
    console.println(prog, "Hello from Aetherix UEFI!", sys_table)

efi_path = app.build("build/uefi_hello.efi")
diskimage.write_disk_image(efi_path.read_bytes(), "build/uefi_hello.img")
```

```
qemu-system-x86_64 -bios /path/to/OVMF.fd -drive
file=build/uefi_hello.img,format=raw
```

Verifying what you built, without trusting it blindly

For the BIOS path, disassemble the boot sector and kernel directly:

```
head -c 512 myos.img > boot.bin
objdump -D -b binary -m i8086 boot.bin
```

For a stronger check that does not just read the bytes but executes them, install Unicorn Engine and use Aetherix's own emulation harness:

```
pip install unicorn
python tests/emulate.py myos.img
```

For UEFI, the equivalent tools are file and objdump for the PE structure and machine code, gdisk and mtools for the GPT/FAT32 disk image, and tests/emulate_uefi.py for real protocol-call execution:

```
file build/uefi_hello.efi
gdisk -l build/uefi_hello.img
python tests/emulate_uefi.py build/uefi_hello.efi
```

18. Using HALs and Protocols

"HAL" and "protocol" mean related but distinct things depending on which side of Aetherix you are working with, and using each effectively means understanding that distinction.

On the BIOS side: the HAL is a driver registry

```
from aetherix import Project

with Project("MyOS") as os:

    @os.kernel_entry
    def main(prog, drivers):
        drivers.keyboard.init(prog)
        drivers.terminal.init(prog, start_row=0)

        drivers.vga.print_string(prog, "Type something:", row=0, col=0)

        prog.label("loop")
        drivers.keyboard.read_char(prog) # blocks; ASCII byte ends up in AL
        drivers.terminal.putchar_reg(prog, regs.AL)
        prog.jump("loop")

    os.build("myos.img")
```

drivers here is the Project's HAL instance, and every attribute access (drivers.vga, drivers.keyboard, drivers.terminal) resolves to a driver module registered under that name. Checking which names are safe to use is one call away: HAL().implemented() versus HAL().scaffolded(), as covered in the HAL chapter.

On the UEFI side: a protocol is a callable interface

UEFI protocols are reached differently: ConOut and ConIn are available directly off the EFI_SYSTEM_TABLE pointer handed to your application, while any other protocol must be located first, by GUID, through Boot Services:

```
from aetherix.uefi import console, keyboard, boot_services, protocol
from aetherix import regs

@app.entry
def main(prog, sys_table):
    console.println(prog, "Reading a key...", sys_table)
    keyboard.read_key(prog, sys_table, regs.R12, regs.R13)
    console.set_cursor_position(prog, sys_table, 0, 5)

    boot_services.locate_protocol(prog, sys_table,
    "9042A9DE-23DC-4A38-96FB-7ADE080516A", regs.R14)
    prog.cmp64(regs.EAX, 0)
    prog.jcc(regs.JZ, "have_protocol")
    console.println(prog, "Protocol unavailable on this firmware.", sys_table)
    prog.jump("end")
    prog.label("have_protocol")
    # ... call methods on the located interface with protocol.call ...
    prog.label("end")
```

The recurring lesson in both cases is the same: always check the result. On the BIOS side, that means checking `HAL().implemented()` before assuming a driver exists; on the UEFI side, it means checking `EAX` (`EFI_STATUS`) after `locate_protocol`, since not every platform implements every protocol, and firmware will tell you plainly if it does not, rather than crashing silently.

19. Extending the Ecosystem: Custom HALs and Protocols

Adding a BIOS-side driver

A driver is a plain Python module whose functions each take a Program (and usually other build-time-known arguments) and append instructions to it — there is no special base class or registration decorator required. Following `vga.py`'s shape:

```
# my_project/serial.py
from aetherix import regs

COM1_PORT = 0x3F8

def write_byte(prog, byte):
    prog.mov8(regs.AL, byte)
    prog.out_al(COM1_PORT)
    return prog
```

```
from aetherix.drivers.hal import HAL
import my_project.serial as serial

hal = HAL()
hal.register("serial", serial)

from aetherix import Project
proj = Project("MyOS", hal=hal)

@proj.kernel_entry
def main(prog, drivers):
    drivers.serial.write_byte(prog, ord('X'))
```

Once a driver is solid and verified, moving it from a project-local registration into `aetherix/drivers/` itself and updating `HAL.__init__` to include it by default is exactly how `vga`, `keyboard`, `speaker`, `terminal`, `graphics`, and `power` all became built-in in the first place. `CONTRIBUTING.md` lists a concrete set of tractable next drivers — a serial port (UART 16550), a parallel-port printer, PCI enumeration — specifically to give new contributors an achievable target.

Adding a new instruction to the native encoder

Sometimes a driver needs an instruction Aetherix's encoder does not have yet. The process is the same regardless of which mode (16/32/64-bit) is involved:

- Add a byte-emitting function to `native/encoder.c`, following the pattern of the existing functions: a plain C function taking the buffer and any operands, appending fixed-width bytes.

- Register its ctypes signature in `aetherix/_native.py`.
- Add a corresponding method on Program in `aetherix/asm.py`.
- Add a test asserting the exact byte sequence against a known-good encoding (cross-checked with objdump or the relevant architecture manual) — never merge an opcode that has not been verified byte-for-byte.

Adding support for a new UEFI protocol

Because `boot_services.locate_protocol()` and `protocol.call()` already cover the general case, adding support for a specific protocol — Graphics Output Protocol, for instance — usually means writing a small, dedicated wrapper module around those two primitives, once the protocol's GUID and the byte offsets of the methods you need are known (both are published in the UEFI specification, and mirrored in any EDK2 header):

```
# my_project/gop.py
from aetherix.uefi import boot_services, protocol
from aetherix import regs

GOP_GUID = "9042A9DE-23DC-4A38-96FB-7ADED080516A"
QUERY_MODE_OFFSET = 0x08
SET_MODE_OFFSET = 0x10

def locate(prog, sys_table, out_reg):
    return boot_services.locate_protocol(prog, sys_table, GOP_GUID, out_reg)

def set_mode(prog, gop_reg, mode_number):
    return protocol.call(prog, gop_reg, SET_MODE_OFFSET, args=[("imm", mode_number)])
```

A real, hardware-verified register-indirect addressing gotcha is worth restating here since it applies directly to writing code like this by hand: RSP and R12 both require a mandatory SIB byte as a base register, and RBP and R13 require an explicit displacement even when it is zero. Aetherix's own `store64/load64` helpers already handle both cases; if you ever hand-encode a ModR/M byte yourself instead of using them, test with a base register from each of those two families before trusting the result.

20. The Aetherix Program Class: A Practical Reference

Program is the single class underlying everything in this book — every BootSector, every Kernel body, every UefiApp entry function is, underneath, a sequence of calls to a Program instance.

Creating one

```
from aetherix.asm import Program

p16 = Program(bits=16) # real mode
p32 = Program(bits=32) # protected mode
p64 = Program(bits=64) # long mode (UEFI)
```

Labels and jumps

Program resolves labels in two passes (as described in the "How Aetherix Works" chapter), so forward references to a label defined later in the same program work correctly, without any special handling from the caller:

```
p = Program(bits=32)
p.label("loop")
p.in_al(0x60)
p.jcc(regs.JZ, "loop") # backward reference -- fine
p.jump("after")
p.label("after") # forward reference from the jmp above -- also fine
```

Raw data alongside code

```
p.db(0x10, 0x00) # a few individual bytes
p.dw(0x1234) # a 16-bit word
p.ascii("Hello", zero_terminate=True)
p.raw_bytes(large_binary_blob) # one bulk node -- use this over db() for anything
# more than a few dozen bytes; db() creates one
# Python object per byte and does not scale
```

Generating unique labels from reusable code

A helper function called more than once in the same program — for instance, once per dispatched key in a keyboard loop — cannot safely use a hardcoded label name, since the same name used twice would collide. `Program.unique_label()` solves this directly:

```
def emit_retry_block(prog):
    label = prog.unique_label("retry") # e.g. "__retry_7", guaranteed fresh
    prog.label(label)
    ...
    prog.jump(label)
```

Referencing your own embedded data

In 16-bit and 32-bit code, `mov16_label()/mov32_label()` compute the absolute address of a label (relative to a base address the surrounding builder, such as Kernel, supplies automatically). In 64-bit code, the equivalent is RIP-relative and needs no base address at all, since it is computed purely from the current instruction pointer:

```
p = Program(bits=64)
p.lea_rip_label(regs.EDX, "my_string") # RDX = absolute address of the label
p.jump("skip")
p.label("my_string")
p.raw_bytes("Hi\x00".encode("utf-16-le"))
p.label("skip")
```

A note on correctness discipline

Every native instruction Program exposes is documented with exactly what it clobbers and any register it must not be given (such as `protocol.call`'s restriction against RAX/RCX as its `table_reg` argument, since those are used internally). Reading a driver function's docstring before reusing its registers for something else is not a formality — it is precisely the kind of check that would have caught more than one of the real bugs described in this book before they shipped.

21. Writing Code with Program: Worked Examples

Example 1: a byte-perfect segment setup

```
from aetherix.asm import Program
from aetherix import regs

p = Program(bits=16)
p.cli()
p.xor16(regs.AX, regs.AX)
p.mov_sreg(regs.DS, regs.AX)
p.mov_sreg(regs.ES, regs.AX)
p.mov_sreg(regs.SS, regs.AX)
p.mov16(regs.SP, 0x7C00)
p.sti()
code = p.assemble()
assert code == bytes([0xFA, 0x31, 0xC0, 0x8E, 0xD8, 0x8E, 0xC0,
0x8E, 0xD0, 0xBC, 0x00, 0x7C, 0xFB])
```

Example 2: a busy-wait delay loop

There is no timer instruction in this encoder subset, so an audible speaker beep, for example, needs an actual delay loop rather than a sleep call — the same "empty for-loop" technique classic BIOS-era code used:

```
def busy_wait(prog, scratch_reg, iterations):
    label = prog.unique_label("delay")
    prog.mov32(scratch_reg, iterations)
    prog.label(label)
    prog.sub32(scratch_reg, 1)
    prog.cmp32(scratch_reg, 0)
    prog.jcc(regs.JNZ, label)
    return prog
```

Example 3: a Shift-aware key dispatch loop

A realistic driver loop needs to distinguish key-press from key-release events (PS/2 Scancode Set 1 marks a release by setting the top bit of the make code), track Shift separately from ordinary character keys, and select between two compile-time-known characters based on that runtime flag:

```

prog.cmp8(regs.AL, SCANCODE_LSHIFT)
prog.jcc(regs.JZ, "shift_on")
prog.cmp8(regs.AL, SCANCODE_LSHIFT | 0x80) # release code = make code | 0x80
prog.jcc(regs.JZ, "shift_off")

prog.test_al(0x80) # any other release code -- ignore it
prog.jcc(regs.JNZ, "loop")

prog.cmp8(regs.AL, SCANCODE_A)
prog.jcc(regs.JNZ, "not_a")
prog.load8(regs.BL, SHIFT_STATE_VAR)
prog.cmp8(regs.BL, 0)
prog.jcc(regs.JNZ, "a_shifted")
prog.mov8(regs.AL, ord('a'))
prog.jump("got_char")
prog.label("a_shifted")
prog.mov8(regs.AL, ord('A'))
prog.jump("got_char")
prog.label("not_a")

```

This exact pattern — generalized across every letter, digit, and punctuation key — is what `aetherix.drivers.keyboard.read_char()` generates, and it was confirmed correct by scripting a real keystroke sequence (Shift+h, i, Shift+1) through a CPU emulator and checking the resulting text on the emulated screen was exactly "Hi!".

Example 4: calling a UEFI protocol method correctly

```

from aetherix.uefi import protocol
from aetherix import regs

# ConOut->SetCursorPosition(This, Column, Row) -- a 3-argument protocol call
prog.load64(regs.ESI, sys_table_reg, 0x40) # RSI = ConOut
protocol.call(prog, regs.ESI, 0x38, args=[("imm", 10), ("imm", 5)])
# Internally: RAX = ConOut->SetCursorPosition; RCX = ConOut (This);
# RDX = 10 (Column); R8 = 5 (Row) -- reserving shadow space and
# restoring the stack around the call automatically.

```

Every one of these examples was built the same way as the rest of this book's claims: written, assembled, disassembled to confirm the bytes, and, where applicable, executed under CPU emulation to confirm the resulting behavior matched intent.

22. Contributing to Aetherix

Aetherix is built on the premise that low-level correctness is worth real verification effort, and contributions are expected to hold to the same standard the project holds itself to.

Where to start

CONTRIBUTING.md lists concrete, ordered next steps rather than an open-ended wish list, specifically so a new contributor has a tractable entry point:

- Extended keyboard scancodes (arrow keys, Insert/Delete/Home/End/Page Up/Down, right Ctrl/Alt) — these send a 0xE0 prefix byte the current driver does not watch for yet.
- A real PIT timer, replacing Program.busy_wait's CPU-cycle-burning delay loop with an actual interrupt-driven one — which also happens to be the real prerequisite for a genuine (non-busy-poll) sleep/wait primitive.
- A serial port (UART 16550) driver — a small, self-contained I/O-port driver, useful immediately for kernel debug logging.
- PCI enumeration — the real prerequisite for any GPU, network, or USB driver work, and useful on its own even before such a driver exists.
- R8–R15 register support was completed during this project's own development specifically to unlock UEFI calls needing a third or fourth argument; a dedicated Graphics Output Protocol wrapper, built on top of the generic locate_protocol/protocol.call mechanism, is a natural next step in the same spirit.
- A full custom UEFI OS loader — calling ExitBootServices, parsing the memory map, and setting up paging and interrupt handling from scratch — is substantially larger than anything above, and deliberately out of scope for the UEFI application support this book describes.

What a native encoder contribution requires

- Every native encoder function must be a fixed-width emission for a given set of argument types, not values — the two-pass label resolver depends on an instruction's size being independent of the operand values it will eventually hold.
- Any new ModR/M-based addressing instruction must account for the RSP/R12 (mandatory-SIB) and RBP/R13 (no-base-without-explicit-displacement) special cases described earlier in this book, and should be tested with a base register from each of those two families, plus a plain register and an R8-range register, before being merged.
- A new opcode needs a test asserting its exact byte sequence, verified against a known-good encoding, not merely "it seems to work".

What a driver or protocol wrapper contribution requires

- Prefer small, testable functions over large monolithic ones, following the shape of the existing drivers.
- Document what real hardware or protocol behavior the code relies on, and what it deliberately does not handle yet — matching the tone of the existing driver source rather than overselling what is implemented.
- Where practical, verify behavior by executing the generated code (CPU emulation) rather than only by disassembling it. Disassembly proves the bytes are plausible; execution proves they do what was intended.

Every one of these rules exists because Aetherix's own development violated it at least once, caught the resulting bug through exactly the verification step being asked for here, and fixed it. That history is documented plainly in the project's own changelog rather than smoothed over, and contributions are expected to add to that record of catching mistakes before they ship, not to skip the step that finds them.

23. License: GNU Affero General Public License v3.0 or later

Aetherix is distributed under the GNU Affero General Public License, version 3, or (at your option) any later version — commonly written as AGPL-3.0-or-later.

What this means in practice

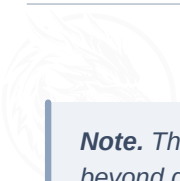
The AGPL is a strong copyleft license, building on the same foundation as the GNU General Public License (GPL), with one specific extension. Under the ordinary GPL, a company can modify GPL-licensed code, run it as a network service, and never be obligated to share those modifications with anyone, because they never "distributed" the software in the traditional sense — only its output, over a network. The AGPL closes that gap: if you run a modified version of AGPL-licensed software and make it available to users over a network, you are obligated to offer those users the corresponding modified source code, on the same terms, just as if you had physically distributed it to them.

The core obligations, summarized

- You may run, study, modify, and redistribute Aetherix, for any purpose, including commercially.
- If you distribute Aetherix, modified or not, you must make the corresponding source code available under the same license.
- If you run a modified version of Aetherix as a network-accessible service, you must offer the source code of your modified version to users interacting with it over the network, not only to users you directly hand a copy of the software.
- Any derivative work must also be licensed under the AGPL (or a later version of it, under the "or later" clause), preserving these same obligations for anyone further down the chain.
- This is a summary for orientation only, not a substitute for the license text itself. The full, authoritative terms are published by the Free Software Foundation and should be consulted directly for anything beyond a general understanding.

Why this license fits a project like Aetherix

A toolkit whose entire value proposition is transparency — showing, rather than hiding, exactly how a boot sector, a kernel, or a UEFI application is constructed at the byte level — is well matched to a license that guarantees the same transparency persists through modification and network deployment. Anyone who improves Aetherix, including by running an improved version as a hosted service, is bound by the same obligation to keep that improvement's source available that the original project itself operates under.



Note. *This chapter is a plain-language summary provided for context, not legal advice. For anything beyond general orientation — in particular, questions about how the AGPL applies to a specific deployment or derivative work — consult the full license text and, where appropriate, qualified legal counsel.*





24. Closing Notes

Aetherix is, deliberately, not a finished operating system, and this book has tried throughout to be precise about the line between what is genuinely working today and what is a documented, honest gap. The BIOS pipeline produces real, verified, bootable images; a handful of drivers cover the hardware that is actually reachable without a bus or protocol stack underneath them; AVFS embeds files without pretending to be a runtime filesystem; and the UEFI pipeline produces a genuine PE32+ application, verified down to booting on real, unmodified firmware.

What is left unbuilt — a real filesystem, a GPU framebuffer, USB, a full custom OS loader that exits boot services and owns the hardware outright — is left unbuilt on purpose, named explicitly rather than glossed over, and pointed at directly as a place for the next contributor to start.

If this book has done its job, you now understand not just what Aetherix does, but why each piece of it works the way it does — down to the individual bytes — and you have everything you need to extend it yourself.